

Design Document

Project Name: Learnly: Etkileşimli Öğrenme Platformu Prototipi **Date:** 20.11.2025

List of Contributors

- Mehmet Gönül (231101027)
- Fikri Mert Sabuncu (231101048)
- Mehmet Emin Asan (231101079)

Table of Contents

- Task Matrix
- System Overview
- Implementation Details
- Use Case Support in Design
- Design Decisions

1. Task Matrix

Task	Responsible Team Member
System Overview & Design Decisions	Fikri Mert Sabuncu
Implementation Details (Backend/Frontend Structure)	Mehmet Gönül
Use Case Support & Database Design Mapping	Mehmet Emin Asan

2. System Overview

Brief Project Description

Learnly, öğretmenlerin öğrenciler için quiz ve sınavlar hazırlayabildiği, öğrencilerin ise bu sınavlara katılıp sonuçlarını takip edebildiği web tabanlı bir etkileşimli öğrenme platformudur. Proje, her kullanıcının kendi rolüyle (öğretmen veya öğrenci) ilişkili verilere erişebilmesini sağlayan katı bir rol tabanlı erişim kontrolü üzerine kurulmuştur.

System Architecture

Sistem, Katmanlı Mimari prensiplerine göre tasarlanmıştır. İstemci tarafı (Frontend) ve sunucu tarafı (Backend) birbirinden bağımsız çalışır ve RESTful API üzerinden iletişim kurar. Veritabanı katmanı, veri

bütünlüğünü sağlamak için ilişkisel bir yapıdadır. Proje bulutta veya yerel sunucuda çalıştırılmak üzere yapılandırılmıştır.

Technology Stack

Projede kullanılan temel teknolojiler şunlardır:

- **Frontend:** React, HTML, CSS, JavaScript
- **Backend:** Java, Spring Boot, Maven
- **Database:** MySQL (Yönetim: DBeaver)
- **Server:** Nginx, Ubuntu
- **Testing:** JUnit, REST Assured, Selenium

Visual Interface

3. Implementation Details

Codebase Structure

Proje, sorumlulukların ayrılığı ilkesine göre modüler klasörlere ayrılmıştır:

- **src/main/java/com/Learny:** Backend kaynak kodları.
 - `Business` : Uygulamanın temel iş kurallarının, veri doğrulamalarının ve hesaplamalarının yapıldığı katmandır. Veri erişim katmanından gelen ham veriyi işleyerek sunum katmanına hazır hale getirir.
 - `DataAccess` : Veritabanı ile uygulama arasındaki bağlantıyı kurarak CRUD (Oluşturma, Okuma, Güncelleme, Silme) işlemlerini yürütür. Hibernate teknolojisi kullanılarak verilerin kalıcı hale getirilmesi bu katmanda sağlanır.
 - `Entities` : Uygulamanın veri modelini oluşturan ve veritabanı tabloları ile birebir eşleşen nesne sınıflarını barındırır. Veri tabanındaki ilişkisel yapının nesne tabanlı programlama tarafındaki karşılığıdır.
 - `restAPI` : İstemciden (Client) gelen HTTP isteklerini (GET, POST vb.) karşılar ve ilgili iş servislerine yönlendirir. İşlenen verileri JSON formatında yanıt olarak dış dünyaya sunar.
- **src/frontend/src:** React frontend kaynak kodları.
 - `src` : Uygulamanın geliştirme sürecindeki tüm kaynak kodlarını, stil dosyalarını ve mantıksal kurguyu barındıran ana çalışma dizinidir.
 - `src/components` : Kullanıcı arayüzünü (UI) oluşturan, modüler ve tekrar kullanılabilir yapı taşlarının bulunduğu klasördür. Her bir dosya, arayüzün belirli bir parçasını temsil eder.
 - `src/api` : Frontend uygulamasının Backend servisleri ile haberleşmesini sağlayan, HTTP isteklerinin (GET, POST vb.) yönetildiği ve servis bağlantılarının yapılandırıldığı katmandır.

Key Implementations

- **Authentication & Authorization:** Spring Security kullanılarak JWT (JSON Web Token) tabanlı kimlik doğrulama uygulanmıştır. Kullanıcı şifreleri `SHA256` ile hashlenerek saklanır.

Component Interfaces (API Endpoints)

Sistemin temel bileşen arayüzleri şunlardır:

1. UserController Base URL: /api

Method	Endpoint	Description	Request Body
GET	/users	Retrieve all users.	N/A
GET	/users/{id}	Retrieve a user by ID.	N/A
POST	/add	Add a new user.	JSON object (User).
POST	/update	Update an existing user.	JSON object (User).
POST	/delete	Delete a user.	JSON object (User).

2. CourseController Base URL: /api/courses

Method	Endpoint	Description	Request Body
GET	/	Retrieve all courses.	N/A
GET	/ {id}	Retrieve a course by ID.	N/A
GET	/teacher/{teacherId}	Retrieve courses taught by a specific teacher.	N/A
POST	/	Create a new course.	JSON object (Course).
PUT	/ {id}	Update a course by ID.	JSON object (Course).
DELETE	/ {id}	Delete a course by ID.	N/A.

3. EnrollmentController Base URL: /api/enrollments

Method	Endpoint	Description	Request Body
GET	/	Retrieve all enrollments.	N/A
GET	/ {id}	Retrieve an enrollment by ID.	N/A
GET	/student/{studentId}	Retrieve enrollments for a specific student.	N/A
GET	/course/{courseId}	Retrieve enrollments for a specific course.	N/A
POST	/	Create a new enrollment.	JSON object (Enrollment).
PUT	/ {id}	Update an enrollment by ID.	JSON object (Enrollment).
DELETE	/ {id}	Delete an enrollment by ID.	N/A.

4. ExamController Base URL: /api/exams

Method	Endpoint	Description	Request Body
GET	/	Retrieve all exams.	N/A
GET	/ {id}	Retrieve an exam by ID.	N/A
GET	/course/{courseId}	Retrieve exams for a specific course.	N/A
POST	/	Create a new exam.	JSON object (Exam).
PUT	/ {id}	Update an exam by ID.	JSON object (Exam).
DELETE	/ {id}	Delete an exam by ID.	N/A.

5. QuestionController Base URL: /api/questions

Method	Endpoint	Description	Request Body
GET	/	Retrieve all questions.	N/A
GET	/ {id}	Retrieve a question by ID.	N/A
GET	/exam/ {examId}	Retrieve questions for a specific exam.	N/A
POST	/	Create a new question.	JSON object (Question).
PUT	/ {id}	Update a question by ID.	JSON object (Question).
DELETE	/ {id}	Delete a question by ID.	N/A.

6. OptionController Base URL: /api/options

Method	Endpoint	Description	Request Body
GET	/	Retrieve all options.	N/A
GET	/ {id}	Retrieve an option by ID.	N/A
GET	/question/ {questionId}	Retrieve options for a specific question.	N/A
POST	/	Create a new option.	JSON object (Option).
PUT	/ {id}	Update an option by ID.	JSON object (Option).
DELETE	/ {id}	Delete an option by ID.	N/A.

7. ExamResultController Base URL: /api/results

Method	Endpoint	Description	Request Body
GET	/	Retrieve all exam results.	N/A
GET	/ {id}	Retrieve an exam result by ID.	N/A
GET	/student/ {studentId}	Retrieve results for a specific student.	N/A
GET	/exam/ {examId}	Retrieve results for a specific exam.	N/A
POST	/	Create a new exam result.	JSON object (ExamResult).
PUT	/ {id}	Update an exam result by ID.	JSON object (ExamResult).
DELETE	/ {id}	Delete an exam result by ID.	N/A.

Visual Interfaces

- **Login Screen:** Kullanıcı adı ve şifre girişi.
- **Exam Information Dashboard:** Eğitimci için sınav oluşturma ve sınav bilgisi görüntüleme. Öğrenci için girdiği ve gireceği sınav bilgilerini görüntüleme.
- **Main Dashboard:** Genel bilgilerin gözüktüğü giriş arayüzü.
- **Exam Interface:** Soru görüntüleme ve seçenek işaretleme.

4. Use Case Support in Design

Aşağıdaki 4 temel kullanım durumu (Use Case), Gereksinim Dokümanı'ndan seçilmiş ve sistem mimarisine entegre edilmiştir.

Use Case 1: Sınav Oluşturma (Create Exam)

- **Requirements:** FG4, FG5.

- **Design:** `ExamController`, eğitmeninden gelen JSON verisini alır. `ExamManager`, sınavı oluşturur ve `HibernateSinavlarDal` aracılığıyla soruları sınav ID'si ile ilişkilendirerek veritabanına kaydeder.

Use Case 2: Sınava Katılım (Take Exam)

- **Requirements:** FG8, FG9.
- **Design:** Öğrenci "Sınava Başla" dediğinde, frontend `/api/exams` endpoint'inden sınavı id'sini çeker, sonra sınav id'sine karşılık gelen soruları `/api/questions/{id}` çeker. Sınav bitiminde cevaplar toplu olarak sunucuya gönderilir ve `GradeService` tarafından anında puanlanır.

Use Case 3: Rol Bazlı Not Görüntüleme - Öğretmen

- **Requirements:** FG6.
- **Design:** Öğretmen, sadece kendi verdiği derslerin (`course_id`) notlarına erişebilir. Backend, isteği yapan kullanıcının o dersin yetkili öğretmeni olup olmadığını kontrol eder.

Use Case 4: Rol Bazlı Not Görüntüleme - Öğrenci

- **Requirements:** FG10.
- **Design:** Öğrenci notlarını sorguladığında, sistem JWT içindeki kullanıcı ID'sini kullanır. SQL sorgusu `WHERE student_id = token_user_id` şeklinde çalıştırılarak veri güvenliği sağlanır.

5. Design Decisions

Technology Comparisons

1. Backend Framework: Spring Boot vs. Node.js

- **Seçim:** Spring Boot
- **Gerekçe:** Ekibin Java konusundaki yetkinliği ve Spring Boot'un sunduğu yerleşik güvenlik modülleri ile tip güvenli yapısı, projenin güvenli ve sürdürülebilir olması için tercih edilmiştir.

2. Database: MySQL vs. MongoDB

- **Seçim:** MySQL
- **Gerekçe:** Projedeki veriler (Öğretmen, Ders, Sınav, Soru, Not) birbirleriyle sıkı ilişkili (structured) verilerdir. Veri bütünlüğünü korumak ve karmaşık sorguları (JOIN işlemleri) performanslı yönetmek için ilişkisel veritabanı (RDBMS) tercih edilmiştir.